

According to a document from **2025-12-15**, the project’s “alpha → policy → go-live” arc is best understood as a sequence of deliberate *runbooked* iterations—where each run tightened (1) evaluation hygiene, (2) objective alignment, and (3) deployment realism.

8. Evolution retrospective

8.1 Phase 0 — Prototype loop: learn “what matters” and instrument everything (Runs ~8–51)

The earliest iterations were not primarily about “winning Sharpe,” but about *making the system measurable*. Runs 8–14 established the first working Kronos-based forecasting backbone and, importantly, surfaced a recurring pattern: **you can get decent calibration/CE without getting consistent IC/Sharpe**, and gates can collapse to “no-trade” unless the evaluator and loss encourage non-degenerate behavior.

The multi-asset era (Runs 25–31+) introduced a “mini-universe” workflow (e.g., MARA/LULU/HIMS plus factor assets like SPY/BTC/VIX) that acted as a *fast design wind tunnel*: asymmetric bins, lag gating, bootstrap-based gate selection, factor-bank loading, stop-grad anneals, and hardened evaluators were all explored in a tightly documented cadence.

A key operational meta-lesson also emerged here: **runbooks that specify exact eval grids, fallback modes, and acceptance criteria are compounding assets**. The runbook explicitly calls out that an evaluator bug (coverage being “hard-clamped”) can invalidate conclusions, and recommends tiered fallback coverage plus bootstrap CIs to avoid brittle gate selection.

What this phase taught us

- If you can’t reproduce “the same run” (config, gates, costs, coverage), you can’t tell whether you learned anything.
 - Much of the early progress came from evaluator/selection discipline as much as from model changes.
-

8.2 Phase 1 — Scaling up: unify the universe and eliminate silent leakage (Runs 101+)

When scaling to the large-universe “full-seq” regime, the first big inflection point was **data hygiene and leakage control**. Run101 explicitly documents two leakage modes:

- **Normalization leakage** (mean/std computed on all splits rather than train-only).

- A **feature exposure leak** (future exposure of features due to an error), which required fixing so the correct shift/masking is applied.

This is a pivotal retrospective point: the project’s credibility depends on treating leakage fixes as first-class “model changes,” because they change the meaning of every metric that follows.

8.3 Phase 2 — Objective evolution: from “proper scoring” to “useful signal” (Runs 110 → 120)

Run110: CRPS-only as a stress test (and a warning) Run110 tested a **CRPS-only** training regime at high weight (“10×”), with padded full-seq. The headline result was that **train IC was positive but did not generalize** (val negative, test near-zero).

The run’s takeaways are explicit: **CRPS is a proper scoring rule for probabilistic forecasts, but it is not an IC objective**; using it as the *primary* objective can improve calibration while degrading rank correlation / tradability.

Run120: reintroduce “decision pressure” (Sharpe term + tails) Run120 responded by explicitly mixing objectives: **CE + CRPS (still high-weighted in this run) + a Sharpe term**, with additional tail emphasis via $w = (1 + 10*|r|)$ to weight large moves more heavily.

This is the second major inflection point in the evolution: it marks a shift from “forecast correctness” toward **portfolio-relevant signal shaping** (even before the dedicated policy model).

8.4 Phase 3 — Architecture evolution: frozen foundation + trainable adapter via gating (Run122 → Run123)

Run122 introduced the now-core pattern of **hybrid encoding**: a frozen/pretrained component plus a lightweight trainable encoder (MLP) combined by a learned gate. The runbook notes a key emergent behavior: the gate mean was **~0.12**, implying the MLP contributed ~12% on average and the system leaned heavily on the pretrained path.

The runbook then proposes a concrete remedy—initializing the gate to prefer the MLP (e.g., `gate_init_prob 0.8`)—and notes that **Run123 is trying this**.

Retrospective interpretation

- This phase wasn’t just “a model tweak”—it codified a scalable design principle: **keep the expensive/time-series foundation stable, and evolve a small, controllable adaptation layer**. The gate statistic

(0.12) is a perfect example of why runbooked diagnostics matter: without it, you might falsely assume the new module is “doing work.”

8.5 Phase 4 — Bridging alpha to portfolio: simple policies as a truth serum (Run125)

Run125 is where the project explicitly validated that the alpha outputs can support *portfolio construction*, using **interpretable baseline policies** on top of frozen signals (run123/epoch_1).

On the **test split (2024Q4)**, Run125 reports:

- **TopK long-only (K=50): Sharpe 0.5788**
- **Linear policy: Sharpe 0.2046**
- **Long/short (K=50): Sharpe -0.6067**

On the “golden” **pre-2019 backtest**, TopK long-only (K=50) shows Sharpe **0.6924**, with an important accounting note that recomputing Sharpe over *all* days (including zero-trade days) yields **~0.428**, highlighting how evaluation definitions (trade-day vs all-day) can materially change reported metrics.

Why this matters in the evolution

- Run125 served as a *sanity bridge*: before investing in a learned allocator, it established that simple, transparent allocators can extract performance from the alpha—while also revealing that **long/short is much less stable** in this regime.
-

8.6 Phase 5 — Learned portfolio policy: adding cross-sectional reasoning and risk controls (Run133)

Run133 formalized the next step: a trainable **cross-sectional policy** evaluated against SPY with costs and constraints, and (in the checkpoint) an additional risk head. The evaluation runbook notes an important implementation detail: the evaluation script loads the policy with `strict=False` and therefore **ignores extra checkpoint keys like `risk_head.*`**.

This is a quintessential “runbook is an asset” moment: the work doesn’t just build the component—it documents that **the evaluator currently does not apply the risk head**, which affects how you interpret any “risk-gated” claims.

8.7 Phase 6 — Go-live: from experiments to a repeatable refresh pipeline (Run134)

Run134 is the handoff from research to a live-ish operating loop. It documents:

- Refreshing Yahoo data through **2025-12-15**.
- Rebuilding sequences and a “fullseq” dataset with **all splits marked live** (RUN100_ALL_SPLITS_TEST=1).
- Reusing **outputs/run123/epoch_1.pt** (no retrain) to dump **2020–2025** signals excluding SPY/QQQ/BTC/VIX.
- Running run133 portfolio evals across epochs 1–50 and selecting **best Sharpe epoch 7 (~1.50)** through 2025-12-15.

A crucial deployment nuance is called out explicitly: **the ticker universe should be pinned to the training manifest**, because adding new tickers without retraining changes cross-ticker attention inputs and can shift trades even with the same checkpoint.

What this phase taught us

- The “real product” is not only a model; it’s a *refreshable, deterministic pipeline* (data → sequences → signals → portfolio eval).
- Cross-ticker models make the universe definition part of the model; “adding ETH” is a model change unless you retrain.

8.8 Meta-lessons from the full evolution

Runbooks compound. The strongest throughline is that each run didn’t just try an idea; it left behind:

- A reproducible command path (train/eval),
- A clear hypothesis,
- A measurable acceptance lens (IC, Sharpe, coverage, costs),
- And explicit “what broke / what we learned” notes.

Calibration ≠ tradability. The CRPS-heavy phase (Run110) crystallized that proper scoring rules can improve distribution quality while hurting rank objectives; Run120 marked the shift toward explicitly including portfolio-relevant terms.

Modularity paid off. The project progressively separated concerns:

- Alpha model produces signals (run123),
- Simple policies validate signal usefulness (run125),
- A learned policy attempts cross-sectional allocation (run133),
- A refresh pipeline operationalizes the stack (run134).

One “next fix” is glaringly high leverage. If the intent is to claim “risk gate + drawdown penalty,” the evaluation and live pipeline should actually apply the risk head (or explicitly state it is currently ignored). The run133 eval notes already document this mismatch—turning that note into a patched evaluator is a natural next evolutionary step.